Documentație Proiect Grafică pe Calculator - Sketchbook Rocket

Autori: Dănilă Tudor-Mihail (Grupa 334), Loghin Antonia-Miruna (Grupa 343)

1. Conceptul Proiectului și Stil Vizual

Proiectul "Sketchbook Rocket" este o aplicație interactivă 2D dezvoltată în limbajul C++ utilizând API-ul grafic OpenGL 3.3 Core Profile, FreeGLUT, GLEW, GLM și parser-ul vectorial NanoSVG.
Aplicația este un joc 2-dimensional în care user-ul conduce o rachetă prin spațiu, pe un caiet școlar. Aceasta adoptă un stil vizual retro-minimalist bazat pe schițe vectoriale preluate direct din fișiere SVG și randate pe un fundal cu o textură de caiet.

- **Mecanici și reguli de joc:**
  - **Decolarea:** Racheta staționează inițial pe sol și își începe zborul la prima apăsare a click-ului stânga de mouse.
  - **Orientarea și Ghidarea:** Racheta se rotește automat cu vârful spre cursorul mouse-ului și înaintează spre acesta.
  - **Modul Turbo și Particule:** La apăsarea tastei SPACE, racheta intră în modul Turbo (primește un impuls temporal), crescându-și viteza și generând un jet dinamic și mobil de particule.
  - **Derularea Mediului:** Fundalul (solul, copacul, soare) coboară pe verticală, iar stelele se repoziționează ciclic, creând iluzia de zbor infinit.
  - **Schimbarea Cromatică ROGVAIV:** Întreaga scenă își schimbă dinamic culoarea prin parcurgerea circulară a celor 7 culori din curcubeu la apăsarea tastelor A și D.

Elemente vizuale din scenă: Rachetă, Soare, Copac, Pământ, Stele și Particule de propulsie.

2. Transformări Geometrice și Matematica Jocului

Transformările sunt realizate utilizând matricea de proiecție ortografică 2D definită prin glm::ortho în intervalul [-1.0, 1.0] x [-1.0, 1.0]:

Transformări principale:

- **Rotația Dinamică:** Unghiul de rotație este calculat folosind funcția atan2(dy, dx) - 1.5708 radieni (pi/2), aliniind vârful sprite-ului cu direcția mouse-ului.
- **Translația și Interpolarea:** Mișcarea rachetei folosește o interpolare liniară între poziția curentă și pozitia cursorului.
- **Emisia Particulelor:** Particulele sunt emise pe vectorul direcțional opus zborului (calculat prin normalizare și offset negativ).
- **Oglindirea și Repoziționarea:** Stelele folosesc o matrice de scalare pe axa X (S = diag(-1, 1, 1)) pentru efectul de mirror (care face parte din codul proiectului dar imaginea aleasă momentan nu este un exemplu potrivit, fiind simetrică orizontal).

3. Elemente de Originalitate și Dificultate Tehnică

- **Parser Vectorial NanoSVG:** Încorporarea fișierelor SVG pentru extragerea coordonatelor și stocarea directă în VAO/VBO-uri la runtime.
- **Sistem Cromatic ROGVAIV în Fragment Shader:** Schimbarea dinamică a culorilor globale se face pe GPU folosind o variabilă uniformă codCol transmisă o singură dată per cadru și evaluată printr-o instrucțiune switch-case.
- **Managementul Particulelor pe CPU/GPU:** Scalare dinamică a particulelor proporțional cu parametrul de viață (life factor).
- **Managementul Obiectelor din scenă:** Translatarea pe veticală a obiectelor din scenă pentru crearea efectului de scrolare și ascensiune și redesenarea periodică a câtorva dintre obiecte.

4. Structură Reprezentativă de Cod

Fragment 1: Schimbarea circulară a culorii (input_management.cpp)

```C#
if (key == 'd' || key == 'D') {
    indexCuloare = (indexCuloare + 1) % 7;
    glutPostRedisplay();
}
if (key == 'a' || key == 'A') {
    indexCuloare = (indexCuloare - 1 + 7) % 7;
    glutPostRedisplay();
}
```

Fragment 2: Transmiterea culorii globale (RenderFunction)

```
C#
glUseProgram(ProgramId);
glUniform1i(codColLocation, indexCuloare);
```

Fragment 3: Selecția ROGVAIV în Fragment Shader (example.frag)

```
C#
switch(codCol) {
    case 0: out_Color = vec4(1.0, 0.0, 0.0, 1.0); break; // Rosu
    case 1: out_Color = vec4(1.0, 0.5, 0.0, 1.0); break; // Oranj
    case 2: out_Color = vec4(1.0, 1.0, 0.0, 1.0); break; // Galben
    case 3: out_Color = vec4(0.0, 1.0, 0.0, 1.0); break; // Verde
    case 4: out_Color = vec4(0.0, 0.4, 1.0, 1.0); break; // Albastru
    case 5: out_Color = vec4(0.29, 0.0, 0.51, 1.0); break; // Indigo
    case 6: out_Color = vec4(0.6, 0.0, 0.8, 1.0); break; // Violet
}
```

Fragment 4: Crearea particulelor în TimerFunction la boost. (renderer.cpp):

```
    for (int i = 0; i < 3; i++) {
        Particula p;
        p.x = rocketX - dirX * offsetDist + ((float)rand() / RAND_MAX - 0.5f) * 0.03f;
        p.y = rocketY - dirY * offsetDist + ((float)rand() / RAND_MAX - 0.5f) * 0.03f;

        float pSpeed = 0.015f + ((float)rand() / RAND_MAX) * 0.01f;
        p.vx = -dirX * pSpeed + ((float)rand() / RAND_MAX - 0.5f) * 0.005f;
        p.vy = -dirY * pSpeed + ((float)rand() / RAND_MAX - 0.5f) * 0.005f;

        p.life = 1.0f;
        particule.push_back(p);
    }
}

for (size_t i = 0; i < particule.size(); i++) {
    particule[i].x += particule[i].vx;
    particule[i].y += particule[i].vy - vitezaBg;
    particule[i].life -= 0.04f;

    if (particule[i].life <= 0.0f) {
        particule.erase(particule.begin() + i);
        i--;
    }
}
```

5. Capturi de Ecran (Rezultate)





6. Contribuții Individuale și Resurse

Proiectul a fost dezvoltat în colaborare directă de către ambii membri ai echipei (Dănilă Tudor-Mihail și Loghin Antonia-Miruna), contribuțiile fiind egale pe toate modulele: parsatul SVG (NanoSVG), arhitectura VAO/VBO, transformările matematice GLM, shaderele GLSL și sistemul de particule.

Resurse utilizate:

- OpenGL 3.3 Core Profile
- GLM (OpenGL Mathematics Library)
- NanoSVG Parser
- FreeGLUT & GLEW